

Scratch থেকে Causal Language Model তৈরি

ভূমিকা

এতদিন পর্যন্ত আমরা বেশিরভাগ ক্ষেত্রে pretrained মডেল নিয়ে নতুন কাজের জন্য fine-tune করেছি — অর্থাৎ pretraining-এর weight পুনরায় ব্যবহার করেছি। Chapter 1-এ যেমন দেখা গেছে, এই পদ্ধতিকে **Transfer Learning** বলা হয়। এটি Transformer মডেলকে বাস্তব কাজে প্রয়োগের সবচেয়ে সফল কৌশল — বিশেষত যখন labeled data কম থাকে।

এই অধ্যায়ে আমরা একটু ভিন্ন পথে হাঁটব। আমরা একটি সম্পূর্ণ নতুন মডেল **scratch** থেকে তৈরি করব।

Scratch থেকে মডেল তৈরি কখন করা উচিত?

এটা তখনই কার্যকর যখন:

- হাতে প্রচুর ডেটা আছে
- সেই ডেটা বিদ্যমান মডেলগুলোর pretraining ডেটা থেকে সম্পূর্ণ আলাদা

তবে একটি language model pretrain করতে শুধু fine-tune করার চেয়ে অনেক বেশি compute resource দরকার।

যে ক্ষেত্রে নতুন মডেল তৈরি করা যুক্তিযুক্ত হতে পারে সেগুলো হলো Musical notes বা সুরের নোটেশন, Molecular sequences যেমন DNA sequence, এবং Programming languages বা source code।

Programming language-এর ক্ষেত্রে এটি সম্প্রতি বিশেষভাবে জনপ্রিয় হয়েছে। **TabNine**, **GitHub Copilot** — OpenAI-এর Codex মডেল দ্বারা চালিত — এই ধরনের tool-ই দীর্ঘ code sequence generate করতে পারে। এই ধরনের **text generation task**-এ সবচেয়ে উপযোগী হলো **auto-regressive** বা **causal language model** — যেমন GPT-2।

এই অধ্যায়ে আমরা কী তৈরি করব?

আমরা একটি **scaled-down code generation** মডেল তৈরি করব। পুরো function বা class লেখার বদলে আমরা এক লাইনের **code completion**-এ মনোযোগ দেব।

মূল focus থাকবে Python-এর **data science stack**-এ, অর্থাৎ **matplotlib**, **seaborn**, **pandas**, এবং **scikit-learn**। এই libraries ব্যবহার করার সময় প্রায়ই নির্দিষ্ট command খুঁজতে হয় — যদি একটি মডেল এই calls autocomplete করে দিতে পারে, তাহলে কাজ অনেক সহজ হয়ে যায়।

Chapter 6-এ আমরা Python source code process করার জন্য একটি efficient tokenizer তৈরি করেছিলাম। এখন সেই tokenizer-কে GitHub repositories থেকে আনা Python code corpus-এ apply করব এবং **Trainer API** ও **Accelerate** দিয়ে মডেল ট্রেন করব।

নোট: এই section-এর code ব্যবহার করে একটি মডেল ট্রেন করে Hub-এ upload করা হয়েছে। সেটি [এখানে](#) পাওয়া যাবে। Text generation-এ randomization থাকায় আপনার output একটু ভিন্ন হতে পারে।

ধাপ ১ — ডেটা সংগ্রহ

Codeparrot Dataset

Python code GitHub-এর মতো code repository থেকে সহজেই পাওয়া যায়। Transformers textbook-এ একটি বড় GPT-2 মডেল pretrain করার জন্য প্রতিটি Python repository scrape করে একটি dataset তৈরি করা হয়েছিল। প্রায় ১৮০ **GB** এবং ২ কোটি **Python** ফাইল সম্বলিত এই GitHub dump-কে **codeparrot** বলা হয়, যা Hugging Face Hub-এ শেয়ার করা হয়েছে।

তবে পুরো corpus-এ ট্রেনিং করা অনেক সময় ও compute নিবিড়। আমাদের শুধু Python data science stack সংক্রান্ত অংশটুকু দরকার। তাই আমরা codeparrot dataset থেকে শুধুমাত্র সেই ফাইলগুলো filter করব যেগুলোতে আমাদের নির্বাচিত libraries-এর অন্তর্গত একটি আছে। পুরো dataset download না করে আমরা **streaming** feature ব্যবহার করে on the fly filter করব।

Keyword Filter Function

```
def any_keyword_in_string(string, keywords):
    for keyword in keywords:
        if keyword in string:
            return True
    return False
```

দুটো উদাহরণে test করা যাক:

```
filters = ["pandas", "sklearn", "matplotlib", "seaborn"]
example_1 = "import numpy as np"
example_2 = "import pandas as pd"

print(
    any_keyword_in_string(example_1, filters), any_keyword_in_string(example_2, filters)
)
```

আউটপুট:

False True

ঠিকঠাক কাজ করছে। এখন এই function ব্যবহার করে streaming dataset filter করার function তৈরি করা যাক:

```
from collections import defaultdict
from tqdm import tqdm
from datasets import Dataset

def filter_streaming_dataset(dataset, filters):
    filtered_dict = defaultdict(list)
    total = 0
    for sample in tqdm(iter(dataset)):
        total += 1
        if any_keyword_in_string(sample["content"], filters):
            for k, v in sample.items():
                filtered_dict[k].append(v)
    print(f'{len(filtered_dict["content"])/total:.2%} of data after filtering.')
    return Dataset.from_dict(filtered_dict)
```

এখন এই function streaming dataset-এ apply করা যাক:

```
# This cell will take a very long time to execute, so you should skip it and go to
# the next one!
from datasets import load_dataset

split = "train" # "valid"
filters = ["pandas", "sklearn", "matplotlib", "seaborn"]

data = load_dataset(f"transformersbook/codeparrot-{split}", split=split, streaming=True)
filtered_data = filter_streaming_dataset(data, filters)
```

আউটপুট:

3.26% of data after filtering.

মূল dataset-এর মাত্র ৩% বাকি থাকলেও এটি বেশ সমৃদ্ধ — ৬ **GB** এবং ৬ লক্ষ **Python script**!

পুরো dataset filter করতে মেশিন ও bandwidth-এর উপর নির্ভর করে ২-৩ ঘন্টা লাগতে পারে। এই দীর্ঘ প্রক্রিয়া এড়াতে আমরা Hub থেকে ইতোমধ্যে filtered dataset সরাসরি download করতে পারি:

```
from datasets import load_dataset, DatasetDict
```

```
ds_train = load_dataset("huggingface-course/codeparrot-ds-train", split="train")
ds_valid = load_dataset("huggingface-course/codeparrot-ds-valid", split="validation")
```

```
raw_datasets = DatasetDict(
    {
        "train": ds_train, # .shuffle().select(range(50000)),
        "valid": ds_valid, # .shuffle().select(range(500))
    }
)
```

```
raw_datasets
```

আউটপুট:

```
DatasetDict({
  train: Dataset({
    features: ['repo_name', 'path', 'copies', 'size', 'content', 'license'],
    num_rows: 606720
  })
  valid: Dataset({
    features: ['repo_name', 'path', 'copies', 'size', 'content', 'license'],
    num_rows: 3322
  })
})
```

পরামর্শ: Language model pretraining অনেক সময় নেয়। প্রথমে উপরের দুটো partial line uncomment করে ডেটার একটি ছোট subset দিয়ে training loop চালিয়ে দেখুন সবকিছু সঠিকভাবে কাজ করছে কিনা এবং মডেল সংরক্ষিত হচ্ছে কিনা। Training loop-এর শেষ ধাপে failure-এর চেয়ে বেদনাদায়ক আর কিছু নেই!

Dataset একটু দেখে নেওয়া

প্রতিটি field-এর প্রথম ২০০ character দেখা যাক:

```
for key in raw_datasets["train"][0]:
    print(f"{key.upper()}: {raw_datasets['train'][0][key][:200]}")
```

আউটপুট:

```
'REPO_NAME: kmike/scikit-learn'
'PATH: sklearn/utils/__init__.py'
```

```
'COPIES: 3'
'SIZE: 10094'
'''CONTENT: '''
The :mod:`sklearn.utils` module includes various utilities.
''''
```

```
from collections import Sequence
```

```
import numpy as np
from scipy.sparse import issparse
import warnings
```

```
from .murmurhash import murm
LICENSE: bsd-3-clause'''
```

`content` field-এ সেই code রয়েছে যেটায় আমাদের মডেল ট্রেন করবে।

ধাপ ২ — Dataset Preprocessing

Context Size নির্ধারণ

প্রথম কাজ হলো ডেটা tokenize করা। যেহেতু আমাদের লক্ষ্য মূলত ছোট function call autocomplete করা, তাই context size অপেক্ষাকৃত ছোট রাখা যাবে। এর সুবিধা হলো মডেল অনেক দ্রুত ট্রেন হবে এবং কম memory লাগবে।

যদি আপনার application-এ বেশি context দরকার হয়, যেমন function definition সহ পুরো ফাইল দেখে unit test লেখা, তাহলে এই সংখ্যা বাড়তে হবে — তবে GPU memory footprint বাড়বে।

এখন আমরা context size ১২৮ **token** নির্ধারণ করব। তুলনামূলকভাবে, GPT-2 ও GPT-3-এ যথাক্রমে ১,০২৪ এবং ২,০৪৮ ব্যবহার করা হয়।

Chunking Strategy

বেশিরভাগ document-এই ১২৮ token-এর বেশি থাকবে। শুধু truncate করলে dataset-এর বড় অংশ নষ্ট হয়ে যাবে।

তাই আমরা `return_overflowing_tokens` option ব্যবহার করে পুরো input tokenize করে একাধিক **chunk**-এ ভাগ করব, ঠিক যেভাবে Chapter 6-এ করা হয়েছিল। সাথে `return_length` option দিয়ে প্রতিটি chunk-এর দৈর্ঘ্য স্বয়ংক্রিয়ভাবে পাওয়া যাবে। শেষের chunk-টি প্রায়ই ছোট হয় — সেগুলো আমরা বাদ দেব, কারণ প্রচুর ডেটা থাকায় এতে padding সমস্যা এড়ানো যাবে।

প্রথম দুটো উদাহরণে দেখা যাক:

```
from transformers import AutoTokenizer

context_length = 128

tokenizer = AutoTokenizer.from_pretrained("huggingface-course/code-search-net-tokenizer")

outputs = tokenizer(
    raw_datasets["train"][:2]["content"],
    truncation=True,
    max_length=context_length,
    return_overflowing_tokens=True,
    return_length=True,
)

print(f"Input IDs length: {len(outputs['input_ids'])}")
print(f"Input chunk lengths: {(outputs['length'])}")
print(f"Chunk mapping: {outputs['overflow_to_sample_mapping']}")
```

Input IDs length: 34
Input chunk lengths: [128, 117, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 41]
Chunk mapping: [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Tokenize Function

```

    if length == context_length:
        input_batch.append(input_ids)
    return {"input_ids": input_batch}

```

```

tokenized_datasets = raw_datasets.map(
    tokenize, batched=True, remove_columns=raw_datasets["train"].column_names
)
tokenized_datasets

```

আউটপুট:

```

DatasetDict({
  train: Dataset({
    features: ['input_ids'],
    num_rows: 16702061
  })
  valid: Dataset({
    features: ['input_ids'],
    num_rows: 93164
  })
})

```

এখন আমাদের কাছে ১২৮ **token**-এর ১ কোটি ৬৭ লক্ষ উদাহরণ রয়েছে — মোট প্রায় ২.১ বিলিয়ন **token**।

তুলনা হিসেবে: OpenAI-এর GPT-3 ও Codex মডেল যথাক্রমে ৩০০ বিলিয়ন ও ১০০ বিলিয়ন **token**-এ ট্রেন করা হয়েছে। আমাদের লক্ষ্য সেই মডেলগুলোর সাথে প্রতিযোগিতা করা নয় — বরং data scientist-দের জন্য একটি দ্রুত **autocomplete tool** তৈরি করা।

Try it out! ছোট context window-এর কারণে ছোট chunk বাদ দেওয়া এখানে বড় সমস্যা নয়। কিন্তু context size বাড়ালে বা corpus ছোট document-এ পরিপূর্ণ হলে বাদ পড়া chunk-এর অংশ বাড়বে। আরও কার্যকর পদ্ধতি হলো batch-এর সব tokenized sample গুলো **eos_token_id** দিয়ে জুড়ে দিয়ে তারপর chunking করা। Exercise হিসেবে **tokenize()** function-টি সেভাবে পরিবর্তন করুন। মনে রাখবেন, **truncation=False** সেট করতে হবে।

ধাপ ৩ — নতুন মডেল **Initialize** করা

Dataset প্রস্তুত, এখন মডেল তৈরির পালা।

আমরা **GPT-2 small** মডেলের মতো একই configuration ব্যবহার করব। Pretrained configuration লোড করে tokenizer size অনুযায়ী vocabulary size ঠিক করব এবং `bos_token_id` ও `eos_token_id` পাস করব:

```
from transformers import AutoTokenizer, GPT2LMHeadModel, AutoConfig
```

```
config = AutoConfig.from_pretrained(
    "gpt2",
    vocab_size=len(tokenizer),
    n_ctx=context_length,
    bos_token_id=tokenizer.bos_token_id,
    eos_token_id=tokenizer.eos_token_id,
)
```

এই configuration দিয়ে নতুন মডেল তৈরি করা যাক। লক্ষ্য করুন, এই প্রথমবারের মতো আমরা `from_pretrained()` ব্যবহার করছি না — কারণ আমরা নিজেরাই একটি মডেল initialize করছি:

```
model = GPT2LMHeadModel(config)
model_size = sum(t.numel() for t in model.parameters())
print(f"GPT-2 size: {model_size/1000**2:.1f}M parameters")
```

আউটপুট:

GPT-2 size: 124.2M parameters

আমাদের মডেলে ১২৪ মিলিয়ন **parameter** আছে যা tune করতে হবে।

ধাপ ৪ — Data Collator তৈরি

Training শুরু করার আগে একটি **data collator** তৈরি করতে হবে, যা batch তৈরির কাজ করবে।

আমরা `DataCollatorForLanguageModeling` ব্যবহার করব — এটি language modeling-এর জন্যই তৈরি। এটি শুধু batch stack ও pad করে না, সাথে **language model labels** তৈরি করে। Causal language modeling-এ input নিজেই label হিসেবে কাজ করে — মাত্র এক ধাপ shift করে। এই collator training চলাকালীন on the fly labels তৈরি করে, তাই `input_ids` duplicate করার দরকার নেই।

`DataCollatorForLanguageModeling` masked language modeling (MLM) ও causal language modeling (CLM) — উভয়ই support করে। Default হলো MLM, কিন্তু `mlm=False` দিয়ে CLM-এ switch করা যায়:


```
from transformers import DataCollatorForLanguageModeling

tokenizer.pad_token = tokenizer.eos_token
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)
```

একটি উদাহরণ দেখা যাক:

```
out = data_collator([tokenized_datasets["train"][i] for i in range(5)])
for key in out:
    print(f"{key} shape: {out[key].shape}")
```

আউটপুট:

```
input_ids shape: torch.Size([5, 128])
attention_mask shape: torch.Size([5, 128])
labels shape: torch.Size([5, 128])
```

উদাহরণগুলো stack হয়েছে এবং সব tensor-এর shape একই।

গুরুত্বপূর্ণ: Input ও label align করার জন্য shifting মডেলের ভেতরে হয়, তাই data collator শুধু input-এর copy হিসেবে label তৈরি করে।

ধাপ ৫ — Trainer দিয়ে মডেল Training

Hub Login

```
from huggingface_hub import notebook_login

notebook_login()
```

Terminal-এ হলে:

```
huggingface-cli login
```

Training Arguments নির্ধারণ

আমরা **cosine learning rate schedule** ব্যবহার করব কিছু warmup সহ এবং effective batch size হবে ২৫৬ — এটি `per_device_train_batch_size × gradient_accumulation_steps` থেকে আসে।

Gradient accumulation তখন ব্যবহার হয় যখন একটি পুরো batch memory-তে ধরে না। এটি বেশ কয়েকটি forward/backward pass-এর মাধ্যমে ধীরে ধীরে gradient তৈরি করে।

```
from transformers import Trainer, TrainingArguments
```

```
args = TrainingArguments(
    output_dir="codeparrot-ds",
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    evaluation_strategy="steps",
    eval_steps=5_000,
    logging_steps=5_000,
    gradient_accumulation_steps=8,
    num_train_epochs=1,
    weight_decay=0.1,
    warmup_steps=1_000,
    lr_scheduler_type="cosine",
    learning_rate=5e-4,
    save_steps=5_000,
    fp16=True,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    tokenizer=tokenizer,
    args=args,
    data_collator=data_collator,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["valid"],
)
```

Training শুরু

```
trainer.train()
```

Training-এ সময় লাগবে — পুরো training set হলে প্রায় ২০ ঘন্টা, আর subset হলে ২ ঘন্টা।

Hub-এ Push করা

```
trainer.push_to_hub()
```

Try it out! মাত্র ৩০ লাইনের code আর **TrainingArguments** দিয়ে raw text থেকে GPT-2 ট্রেনিং পর্যন্ত পৌঁছানো গেল। নিজের dataset দিয়ে চেষ্টা করুন।

টিপ: একাধিক GPU সম্পন্ন machine-এ access থাকলে সেখানে চালান। Trainer স্বয়ংক্রিয়ভাবে multiple machine manage করে, এতে training অনেক দ্রুত হয়।

ধাপ ৬ — Pipeline দিয়ে Code Generation পরীক্ষা

এখন আসল পরীক্ষার সময়। মডেল কতটা ভালো কাজ করছে দেখা যাক।

প্রথমে মডেলকে একটি **text generation pipeline**-এ wrap করি এবং দ্রুত generation-এর জন্য GPU-তে রাখি, যদি পাওয়া যায়:

```
import torch
from transformers import pipeline

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
pipe = pipeline(
    "text-generation", model="huggingface-course/codeparrot-ds", device=device
)
```

পরীক্ষা ১ — Scatter Plot তৈরি

```
txt = ""
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create scatter plot with x, y
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

আউটপুট:

```
# create some data
x = np.random.randn(100)
y = np.random.randn(100)
```

```
# create scatter plot with x, y
plt.scatter(x, y)
```

```
# create scatter
```

ফলাফল একদম সঠিক। `plt.scatter(x, y)` — ঠিক যা দরকার।

পরীক্ষা ২ — Pandas DataFrame তৈরি

```
txt = ""
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create dataframe from x and y
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

আউটপুট:

```
# create some data
x = np.random.randn(100)
y = np.random.randn(100)

# create dataframe from x and y
df = pd.DataFrame({'x': x, 'y': y})
df.insert(0, 'x', x)
for
```

সঠিক উত্তর এসেছে — `pd.DataFrame({'x': x, 'y': y})`। তবে এরপর `x` column আবার insert করেছে এবং generate token সীমিত থাকায় `for` loop অসম্পূর্ণ রয়ে গেছে।

পরীক্ষা ৩ — GroupBy Operation

```
txt = ""
# dataframe with profession, income and name
df = pd.DataFrame({'profession': x, 'income': y, 'name': z})

# calculate the mean income per profession
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

আউটপুট:

```
# dataframe with profession, income and name
df = pd.DataFrame({'profession': x, 'income': y, 'name': z})

# calculate the mean income per profession
profession = df.groupby(['profession']).mean()

# compute the
```

চমৎকার! `df.groupby(['profession']).mean()` — একদম সঠিক পদ্ধতি।

পরীক্ষা ৪ — Scikit-learn Random Forest

```
txt = ""
# import random forest regressor from scikit-learn
from sklearn.ensemble import RandomForestRegressor

# fit random forest model with 300 estimators on X, y:
"""
print(pipe(txt, num_return_sequences=1)[0]["generated_text"])
```

আউটপুট:

```
# import random forest regressor from scikit-learn
from sklearn.ensemble import RandomForestRegressor

# fit random forest model with 300 estimators on X, y:
rf = RandomForestRegressor(n_estimators=300, random_state=random_state, max_depth=3)
rf.fit(X, y)
rf
```

দারুণ! `RandomForestRegressor` সঠিকভাবে initialize করে `fit` করেছে।

ফলাফলের মূল্যায়ন

এই কয়েকটি উদাহরণ দেখে মনে হচ্ছে মডেলটি Python data science stack-এর syntax অনেকটাই শিখে নিয়েছে। অবশ্য real-world deployment-এর আগে আরও পুঙ্খানুপুঙ্খ evaluation দরকার।

কখনও কখনও নির্দিষ্ট use case-এর জন্য মডেল ট্রেনিংয়ে আরও customization প্রয়োজন হয়। যেমন batch size dynamically update করা বা bad examples skip করার জন্য conditional training loop। এই ধরনের কাজের

জন্য **Trainer** subclass করা বা scratch থেকে training loop লেখার দরকার হতে পারে — আর সেখানেই **Accelerate** কাজে আসে।